

Explicação Linha por Linha

`Pilha *novo = new Pilha();` Aloca dinamicamente um novo nó ("caixinha") na memória do computador para guardar o novo registro.

`novo->Nome = N;` e `novo->Telefone = Tel;` Preenche as variáveis desse novo nó com os dados de Nome e Telefone passados por parâmetro.

`novo->elo = T;` Faz o ponteiro de ligação (`elo`) do novo nó apontar para onde o atual topo `T` aponta. Se a pilha estava vazia, ele aponta para `NULL`. Se já continha elementos, ele se conecta ao elemento que até então estava no topo.

`return novo;` Retorna o endereço deste novo nó. No arquivo `main.cpp`, a linha `topo = P.InserirPilha(topo, n, t);` captura esse retorno, fazendo com que a variável `topo` seja atualizada para o endereço do elemento recém-criado.

2. Inserção na Fila (`Fila::Inserir`): A fila segue a regra **FIFO** (*First In, First Out* - O primeiro que entra é o primeiro que sai). Elementos novos sempre entram no **Fim** da fila, enquanto as remoções ocorrem no **Início**.

Explicação Linha por Linha

`Fila *novo = new Fila();` Cria um novo nó na memória para armazenar o nome da pessoa da vez.

`novo->Nome = N;` Copia o nome recebido para dentro do novo nó.

`novo->elo = NULL;` Como em uma fila os novos elementos são inseridos sempre no final, **o último elemento nunca tem ninguém atrás dele**. Por isso, o `elo` do novo nó obrigatoriamente aponta para `NULL`.

`if(F != NULL) { F->elo = novo; }` O ponteiro `F` representa o atual **Fim** da fila. Se a fila **não** estiver vazia (`F != NULL`), o elemento que costumava ser o último da fila (`F->elo`) agora precisa apontar para o `novo` nó que acabou de chegar.

`F = novo;` e `return F;` A variável local `F` é atualizada para apontar para o `novo` nó (já que ele se tornou o novo fim da fila) e esse endereço é retornado para atualizar o ponteiro `Fim` lá no `main.cpp` (`Fim = F.Inserir(Fim, nome);`).

Se a fila estiver vazia: O primeiro elemento inserido passa a ser, ao mesmo tempo, o **Início** e o **Fim** da fila.

Se a fila já tiver elementos: Apenas o ponteiro `Fim` se move para o novo nó, mantendo o ponteiro `Início` intacto no primeiro elemento que chegou.

1.1 Implementação da Estrutura de Dados do tipo Pilha

1.1.1 Pilha.h

```
#include <iostream>

class Pilha{
private:
    string Nome, Telefone;
    Pilha *elo;
public:
    Pilha* InserirPilha(Pilha*, std::string, std::string);
    void PercorrerPilha(Pilha*);
    Pilha* RemoverPilha(Pilha*);
    Pilha* EsvaziarPilha(Pilha*);
};
```

1.1.2 Pilha.cpp

```
#include "Pilha.h"

Pilha* Pilha::InserirPilha(Pilha *T, std::string N, std::string Tel){
    Pilha *novo = new Pilha();
    novo->Nome = N;
    novo->Telefone = Tel;
    novo->elo = T;
    return novo;
};

void Pilha::PercorrerPilha(Pilha *T){
    Pilha *aux = T;
    while(aux != NULL){
        std::cout << aux->Nome << " - " << aux->Telefone << std::endl;
        aux = aux->elo;
    }
};

Pilha* Pilha::RemoverPilha(Pilha *T){
    Pilha *aux = T;
    T = T->elo;
    delete(aux);
    return T;
};

Pilha* Pilha::EsvaziarPilha(Pilha *T){
    Pilha *aux = T;
    while(aux != NULL){
        T = T->elo;
        delete(aux);
        aux = T;
    }
    return T;
};
```

1.1.3 main.cpp

```
#include "Pilha.h"

using namespace std;

void Menu(){
    system("clear");
    cout << "1 - Inserir dados\n";
    cout << "2 - Exibir dados\n";
    cout << "3 - Apagar registro\n";
    cout << "4 - Esvaziar pilha de dados\n";
    cout << "5 - Finalizar programa\n";
    cout << "Informe sua opção: ";
};

int main()
{
    Pilha P, *topo = NULL;
    int op;
    string n, t;
    do{
        Menu();
        cin >> op;
        switch(op){
            case 1:
                cout << "\nDigite o nome: ";
                cin.ignore(); // limpa o buffer
                getline(cin,n); // armazena até digitar o enter
                cout << "\nDigite o telefone: ";
                getline(cin,t);
                topo = P.InserirPilha(topo, n, t);
                cout << "\nRegistro incluído com sucesso!!\n";
                break;
            case 2:
                P.PercorrerPilha(topo);
                break;
            case 3:
                if(topo == NULL){
                    cout << "\nSem registros para deletar!\n";
                }else{
                    topo = P.RemoverPilha(topo);
                    cout << "\nRegistro deletado!\n";
                }
                break;
            case 4:
                if(topo == NULL){
                    cout << "\nSem registros para deletar!\n";
                }else{
                    topo = P.EsvaziarPilha(topo);
                    cout << "\nPilha vazia!\n";
                }
                break;
            case 5:
                cout << "\nTchau!!\n";
                break;

            default:
                cout << "\nOpção inválida!\n";
        }
        cout << "\nPressione Enter para continuar!! ";
        cin.ignore().get();
    }while(op != 5);

    return 0;
}
```

1.1 Implementação da Estrutura de Dados do

1.1.1 Fila.h

```
#include <iostream>
class Fila
{
    public:
        std::string Nome;
        Fila *elo;
        Fila* Inserir(Fila*, std::string);
        void Listar(Fila*);
        Fila* Excluir(Fila*);
        Fila* Esvaziar(Fila*);
};
```

1.2 Fila.cpp

```
#include "Fila.h"

Fila* Fila::Inserir(Fila *F, std::string N){
    Fila *novo = new Fila();
    novo->Nome = N;
    novo->elo = NULL;
    if(F != NULL){
        F->elo = novo;
    }
    F = novo;
    return F;
};

void Fila::Listar(Fila *I){
    Fila *aux = I;
    while(aux != NULL){
        std::cout << aux->Nome << "\n";
        aux = aux->elo;
    }
};

Fila* Fila::Excluir(Fila *I){
    Fila *aux = I;
    I = I->elo;
    delete(aux);
    return I;
};

Fila* Fila::Esvaziar(Fila *I){
    Fila *aux = I;
    while(aux != NULL){
        I = I->elo;
        delete(aux);
        aux = I;
    }
    return I;
};
```

1.2.1 main.cpp

```
#include "Fila.h"

using namespace std;

void Menu(){
    system("clear");
    cout << "1 - Inserir\n";
    cout << "2 - Listar\n";
    cout << "3 - Excluir\n";
    cout << "4 - Esvaziar\n";
    cout << "5 - Sair\n";
    cout << "Escolha sua opção: ";
};

int main()
{
    Fila *Inicio = NULL, *Fim = NULL, F;
    string nome;
    int op;
    do{
        Menu();
        cin >> op;
        switch(op){
            case 1:
                cout << "Informe o nome: ";
                cin.ignore();
                getline(cin, nome);
                Fim = F.Inserir(Fim, nome);
                if(Inicio == NULL)
                    Inicio = Fim;
                cout << "\nInserido!\n\n";
                break;
            case 2:
                if(Inicio == NULL)
                    cout << "Fila vazia!\n";
                else{
                    cout << "\nCadastrados\n\n";
                    F.Listar(Inicio);
                }
                break;
            case 3:
                if(Inicio == NULL)
                    cout << "Fila vazia!\n";
                else{
                    Inicio = F.Excluir(Inicio);
                    cout << "\nExcluido!\n\n";
                }
                break;
            case 4:
                if(Inicio == NULL)
                    cout << "Fila vazia!\n";
                else{
                    Inicio = F.Esvaziar(Inicio);
                    Fim = Inicio;
                }
                break;
            case 5:
                break;
        }
    } while(op != 5);
}
```

```
        cout << "\nFila esvaziada!\n\n";
    }
    break;
case 5:
    cout << "\nFim!\n\n";
    break;
default:
    cout << "Inválido!\n\n";
}
cin.ignore().get();
}while(op != 5);
return 0;
}
```